Computers & Security 154 (2025) 104432

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Performance analysis of dynamic ABAC systems using a queuing theoretic framework

Gaurav Madkaikar [a,1], Karthikeya S.M. Yelisetty [a,1], Shamik Sural [a], Jaideep Vaidya [b,*], Vijayalakshmi Atluri [b]

[a] Indian Institute of Technology, Kharagpur, West Bengal, India
[b] Rutgers University, Newark, NJ, USA

ARTICLE INFO

Keywords:
Attribute-based access control
Policy mining
Incremental authorization
Queue with vacation

ABSTRACT

A policy comprised of a set of rules forms the backbone of Attribute-based Access Control (ABAC) systems. Every incoming request is checked against such a policy and if at least one rule grants the access, it is allowed. Else, access is denied. The initial ABAC policy could be hand crafted by the security administrator or mined from a given set of authorizations using a policy engineering technique. In dynamic ABAC systems, over a period of time, additional authorizations may have to be granted or some removed as per situational changes. These changes are maintained in an auxiliary list. For access resolution, both the policy as well as the auxiliary list are considered before taking a decision. Since such a list can grow indefinitely and checking it adversely affects access resolution efficiency, periodic policy rebuilding must be done by combining the existing policy and the auxiliary list. However, regenerating the ABAC policy requires re-running computationally expensive policy mining algorithms. Further, access mediation has to be put on hold while this step is being carried out, resulting in periods of unavailability of the system. In this paper, we study the intricate problem of balancing access request resolution, accommodating dynamic authorization updates, and ABAC policy rebuilding. We employ a queuing theoretic approach where the access mediation process is modeled as an M/G/1 queue with vacation or limited service. While the server is primarily involved in resolving access requests, it occasionally goes on vacation to rebuild the ABAC policy. We study the effect of queue discipline on several performance parameters like request arrival rate, access resolution time, vacation duration and interval between vacations. Results of an extensive set of experiments provide a direction toward efficient implementation of dynamic ABAC systems.

1. Introduction

Access control mechanisms constitute a critical component of computer security as these are central to maintaining security, integrity and functionality of information systems and resources. They refer to specific methods for enforcing a set of rules defining who can access what data and other resources, commonly known as the access control policy for the organization. Access control models guide the design and implementation of mechanisms, specifying how access control decisions are made and policies are structured. Some of the more commonly used models include Discretionary Access Control (DAC) (Harrison et al., 1976), Mandatory Access Control (MAC) (Sandhu, 1993) and Role-based Access Control (RBAC) (Sandhu et al., 1996). In DAC, accesses are allowed at the discretion of the resource owner and such a set is typically maintained as an access control matrix (ACM). In contrast, MAC enforces strict security policies based on pre-defined clearance and classification levels, which is primarily suited for government or military organizations. In RBAC, access permissions are tied to the notion of a role, and users are assigned to one or more roles based on their job functions. RBAC simplifies access management by grouping users with similar access needs and assigning appropriate permissions rather than granting access directly to individual users. It provides considerable flexibility in situations involving a set of users with well-defined roles, but fails in dynamic environments where the same is not known *a priori*. Attribute-based Access Control (ABAC) (Hu et al., 2013) was proposed more recently to overcome some of the shortcomings of these models. In ABAC, access requests are resolved based

* Corresponding author.
E-mail addresses: gmadkaikar@iitkgp.ac.in (G. Madkaikar), yk553@kgpian.iitkgp.ac.in (K.S.M. Yelisetty), shamik@cse.iitkgp.ac.in (S. Sural), jsvaidya@rutgers.edu (J. Vaidya), atluri@rutgers.edu (V. Atluri).

[1] Gaurav Madkaikar and Karthikeya S.M. Yelisetty made equal contributions to the paper.

https://doi.org/10.1016/j.cose.2025.104432
Received 22 October 2024; Received in revised form 7 March 2025; Accepted 10 March 2025
Available online 20 March 2025
0167-4048/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

on an appropriate set of rules (collectively called the ABAC Policy), each defined in terms of the attributes of subjects, resources and the environmental context. There are several application domains in which ABAC can be meaningfully used (Kaur et al., 2024; Bakhtiary et al., 2024; Mahalingam and Sharma, 2024; Gupta et al., 2023).

Due to its versatility, there has been a growing interest among enterprises to migrate from traditional access control models to ABAC. The current body of research in ABAC addresses several concerns associated with this process of migration including enforcement efficiency, scalability and application-specific intricacies. The policy engineering techniques that construct an ABAC policy from existing accesses in an organization are referred to as policy mining. Since manually constructing an appropriate ABAC policy covering all the existing accesses is both time consuming and costly, several algorithmic methods have been proposed for policy mining (Xu and Stoller, 2015; Aggarwal and Sural, 2023; Abu Jabal et al., 2020; Jabal et al., 2023). Once an ABAC system becomes operational, the ABAC policy rules may change due to the changing organizational needs and situations, resulting in new authorizations or removal of the existing ones. However, there is little attempt yet to systematically study the effect of dynamic changes in access control requirements on performance except for an initial work recently done by us (Madkaikar et al., 2024). The existing literature on ABAC either focuses on developing policy mining algorithms for a pre-specified set of authorizations or on efficient evaluation of the policy rules. The interplay between the two when new authorizations get dynamically added, cannot be handled by them.

In this paper, we deliberate on the impact of performing policy mining for enterprises having an evolving policy, with particular emphasis on efficient server utilization and system response. As new authorizations get added to or removed from the system, the updates are temporarily stored in an *Auxiliary List* to facilitate access mediation without changing the existing policy. Consolidation is done periodically at appropriate intervals. We model such an ABAC system as an M/G/1 queue with vacation (Bertsekas and Gallager, 1992; Stewart, 2009), where the server operates in two modes, namely, *Access Resolution* and *Policy Mining*, effectively balancing access control enforcement and mining of an updated policy. Two queuing disciplines have been studied - M/G/1 Queue with Exhaustive Service and Single Vacation, and M/G/1 Queue with Exhaustive and Limited Service system. Based on this queuing theoretic framework, we have implemented a system designed on the client–server architecture, where the server handles concurrent client requests using multi-threading. To assess its performance, an extensive series of experiments were conducted on datasets having a diverse set of attributes. We present a detailed analysis of the observed queuing parameter values, including response latency and server utilization. The findings help to identify the ABAC system design considerations that have higher impact on its performance. Information security administrators can carry out similar studies to determine the bottlenecks in their dynamic ABAC systems and resolve those systematically.

The main contributions of the paper are summarized below:

i We present a novel queuing theoretic framework for performance analysis of dynamic ABAC systems.
ii Two variants of *M/G/1 queue with vacation* models have been used for studying the impact of access request arrival rate and access resolution rate, specifically when incremental changes in the authorization list are allowed to happen in the system thereby demanding periodic restructuring of the ABAC policy.
iii A realistic multi-threaded test bed based on the client server architecture has been set up for conducting extensive experiments that show the utility of our framework. This test bed will be useful for additional work in this area.

The rest of the paper is structured as follows. In Section 2, we describe some of the foundational concepts useful for understanding the rest of the article. Related work is reviewed in Section 3. Section 4 presents the proposed method for modeling dynamic ABAC systems using queues with vacation. System level considerations for implementing the approach are detailed in Section 5. We present the results from an extensive set of experiments in Section 6. Finally, Section 7 concludes the article and presents directions for future research.

Table 1
Sample ABAC policy.

Rule	Role	Department	Type	Time	Operation
r_1	Nurse	Cardiac	Medical	Work	Update
r_2	Doctor	Paediatric	Medical	*	Update
r_3	Doctor	*	Medical	Non-work	Read
r_4	Admin	General	Personal	Work	Update

2. Preliminaries

In this section, we provide an overview of ABAC and introduce the problem of policy mining, along with a brief review of the various algorithms used for the same.

2.1. Attribute-based access control

Attribute-based Access Control defines access rights depending on user, object and environmental attributes. Any ABAC system is comprised of four main components, namely, attributes of users (UA), objects (OA) and environment (EA), a policy Π, a policy decision point (PDP) and a policy enforcement point (PEP) (Hu et al., 2013). Attributes are characteristics of an entity or an event like user designation, department, object type, security classification, time and location of access, etc., while the policy defines a set of rules governing accesses based on such attributes. The PDP evaluates the policy for every access request, making access decisions, while the PEP enforces these decisions by allowing or denying the access accordingly. Thus, any authorization in ABAC, e.g., user *Alice* can *read* a file *F1* during *Working Hours*, is specified in terms of the attributes of Alice, those of F1, and a rule in the organizational policy Π that is valid during the working hours.

An ABAC Policy Π is a union of access rules, each rule being a conjunction of attribute–value pairs. A rule$r \in \Pi$ is a 4-tuple of the form $<SC, OC, EC, op>$, where each of *SC*, *OC* and *EC* represents a set of subject, object and environmental attribute–value pairs, respectively. *op* is any active operation like *read*, *write*, *execute*, etc. The set of all possible operations is *OP*. An access request can be considered to be a 3-tuple $<u, o, op>$denoting that the user *u* intends to carry out operation *op* on the object *o*. Such a request is resolved by matching the attribute values of the requesting user and the requested object to those of the rules in the policy. Access is granted if at least one rule in Π satisfies the attribute values and the current environmental conditions are met for that rule, else the requested access is denied.

Example 1. Consider a hospital where access to patient records is governed by the following ABAC system.

- UA: {role, department}
- OA: {type}
- EA: {time}
- OP: {Read, Update, Delete}

Let each of the subject, object and environmental attributes stated above, take discrete categorical values and their domains are as follows:

- V^U_{role} = {Doctor, Nurse, Admin}
- $V^U_{department}$ = {General, Paediatric, Cardiac}
- V^O_{type} = {Medical, Personal}
- V^E_{time} = {Work, Non-work}

The policy Π of this illustrative ABAC system consists of the rules shown in Table 1. For the sake of brevity, we build rules using only the '=' operator to connect an attribute with its corresponding value. A '*' denotes a wild card matching any possible value. Based on these rules, if a user u, who is a pediatric doctor, requests to update or read a resource r, which is a medical record, during non-working hours, they will be allowed to do so at any time, under rules r_2 and r_3, respectively.

2.2. ABAC policy mining

Policy mining aims to automate the generation of ABAC policies by analyzing existing system configurations or logs of user activities that are permitted in the system — collectively called the set of authorizations (Talukdar et al., 2017; Xu and Stoller, 2015). This is particularly useful when transitioning from traditional access models like DAC or RBAC to ABAC, since explicit policies that reflect current access patterns can be defined without proceeding *ab initio*. As an organization grows, policy-mining techniques can scale effectively to handle increasing amounts of data and complex access patterns. It can result in more optimized policies by eliminating redundant or overly permissive rules, leading to a leaner and efficient access control system. Formally, the policy mining problem can be stated as follows.

Policy mining problem: *Given a set of users U, a set of objects O, a set of user attributes UA, a set of object attributes OA, two sets U_v and O_v containing all the user–user attribute value pairs and object–object attribute value pairs, and a set of authorizations A, derive an ABAC policy Π such that the same set of accesses is provided by the policy and the number of rules in Π is minimized.*

ABAC policy mining is an active area of research with diverse approaches proposed that aim to extract pertinent information for a transition from traditional access control models (Abu Jabal et al., 2020). Initially, rule-based extraction is done that involves parsing of existing rules to obtain key elements like users, objects, actions and conditions. Attribute identification focuses on analyzing various data sources, including user profiles and resource metadata, to define attributes in the ABAC model. Deep Learning and Natural Language Processing techniques can be employed to process and extract insights from text-based access control policies (Mocanu et al., 2015). Log analysis considers reviewing access logs and audit trails to understand the current access decision mechanisms.

Automated policy inferencing utilizes inference techniques to discover relationships between entities based on access decisions. Both bottom up (Das et al., 2018b; Talukdar et al., 2017) as well as hybrid approaches (Das et al., 2018a) have been introduced in the literature. Xu and Stoller (2015) proposed one of the first algorithms for mining ABAC policies from access control lists (ACLs) and entity attribute data. Experiments with various case studies and synthesized datasets have shown the effectiveness of this algorithm, making it a pertinent choice for use in our work.

3. Related work

In this section, we review existing literature that relates to our work.

Despite its flexibility, the dynamic nature of ABAC poses significant challenges in policy management (Batra et al., 2019), especially as systems scale and evolve. Burmester et al. (2013) introduce T-ABAC, an extension of traditional ABAC that incorporates real-time attributes for highly dynamic systems, where availability is critical. Their model is particularly useful in time-sensitive environments such as substation automation and medical cyber–physical systems — typical use cases for which traditional ABAC models fail to address real-time constraints effectively. By integrating real-time attributes into access control decisions, T-ABAC enhances both security and system responsiveness.

Policy mining becomes a critical tool in this context, helping administrators distill and optimize access control rules from vast amounts of data, ensuring that ABAC rules remain effective and manageable over time. The first major mining algorithm for ABAC policies was introduced by Xu and Stoller (2015). However, their work supports only those authorizations that grant access to resources. Moreover, the policies are comprised of rules having varied sizes, resulting in increased resolution time. Considering this, Gautam et al. (2017) propose an improvement by imposing a strict constraint on the allowed number of attributes while defining a rule. For quite some time, the available mining algorithms failed to consider negative rules, until Iyer and Masoumzadeh (2018) addressed this problem in their work.

However, none of the initial work on policy mining considered one of the main strong points of ABAC — the availability of defining environmental context. Das et al. (2018b) proposed a Gini impurity-based policy mining algorithm that mines both positive and negative rules, including environmental attributes. Aggarwal and Sural (2023) approached the problem of policy mining by treating it as a search problem, rather than relying on heuristics for covering the input set of authorizations. To this effect, they employed a randomized algorithm named RanSAM for selecting the best policy from a valid policy space. With new developments in the machine learning field, Mocanu et al. (2015) proposed a novel policy mining approach using deep learning models exclusively relying on existing access logs.

Recent studies have further advanced the understanding of ABAC by addressing critical gaps in dynamic analysis and real-time access control. Schlegel and Amthor (2021) introduce DABAC, a formal modeling scheme designed to improve the dynamic analysis of ABAC policies. Their approach enables a more structured reasoning framework, bridging the gap between policy specification and enforcement. Xia et al. (2022) propose an automated technique for extracting ABAC policies from natural language documents using deep learning models like BERT and semantic role labeling. This method significantly improves policy accuracy and reduces manual effort, particularly in complex domains such as healthcare systems.

In an alternative approach, Alohaly et al. (2022) explore the integration of cyber deception techniques with ABAC for enhancing insider threat detection, introducing an adaptive security model that dynamically modifies access control policies based on the detected threats. Abouzaid et al. (2019) propose a queuing-theoretic framework for flying mesh network-assisted IoT environments, modeling drone-assisted network access as a queuing system to analyze performance stability. This framework provides insights into optimizing resource allocation in dynamic access control scenarios. Furthermore, a recent work by Son et al. (2016) introduce Rew-XAC, an approach that adapts ABAC enforcement dynamically based on evolving policies, significantly improving real-time decision-making.

There has also been extensive work on analyzing temporal access control models (Uzun et al., 2012) and addressing the challenges associated with migration from traditional access control models to ABAC. Abu Jabal et al. (2020) propose a novel framework named Polisma that incorporates contextual information in specifying policies by leveraging data mining and machine learning techniques. Talegaon et al. (2022) address the problem of dynamic updates to an ABAC policy without interrupting access mediation. Other relevant work in this field includes use of hierarchical clustering (Shang et al., 2024) and biclique analysis (Perez-Haro and Diaz-Perez, 2024).

So far, there is only limited work in the literature that consider efficient evaluation of ABAC policies. While the PolTree approach proposed by Nath et al. (2019) is one of the first attempts in this direction, Paul and Sural (2021) suggest a novel method using high dimensional indexing techniques that supports selective relaxation. In contrast, a Binary Decision Diagram based approach was introduced by Morisset et al. (2019). They use query constraints and attribute value power for avoiding evaluating queries that do not constitute a valid system state, and for identifying which attribute values should be considered for evaluation.

The concept of applying queuing models to practical scenarios was first addressed in service systems, where servers are not continuously available (Doshi, 1986). These approaches are vital in environments with highly variable access patterns, such as cloud computing platforms and IoT systems. There, the ability to accurately process access requests directly impacts security and user experience (Menasce, 2000). While no prior work has specifically addressed the domain of access control using queuing models, it has often been used to evaluate the performance of networks and cloud systems (Harchol-Balter, 2013). Moreover, the work proposed in Goyal et al. (2022) provides a practical implementation of ABAC enforcement in smart home IoT systems.

The M/G/1 queue model with vacation introduces a powerful framework for analyzing service systems where servers may take breaks or vacations during idle periods (Shortle et al., 2018). For example, Takagi (1991) extensively analyzed vacation models for queues, providing a framework for evaluating system performance metrics. A more specific definition of M/G/1 queues with a single vacation policy is explored in Bertsekas and Gallager (1992), where server vacations are considered exclusively in case of an empty access request queue. Incorporating vacation models in real-world applications allows for a more nuanced understanding of system behavior, especially in scenarios where service channels require periodic maintenance (Doshi, 1986). Further exploration of such queuing models led to variations considering multiple vacation types and non-exhaustive service disciplines as described in Tian and Zhang (2006).

Among related fields of application, Ma and Fan (2022) use queuing models to evaluate Practical Byzantine Fault Tolerant (PBFT) consensus processes in blockchain systems, proving queuing theory can be used for security assessments. Trenchev et al. (2023) use mathematical modeling, including queuing systems, to evaluate cyber security performance. In contrast, Kumar et al. (2021) use queuing theory to optimize cloud service availability and security with a linked job reneging model. Mas et al. (2022) developed a fog computing queuing model that efficiently distributes workloads under dynamic system settings, which is relevant to access control mechanisms handling changing request loads.

Queuing theory has also been applied in real-world scheduling problems. For example, Ala et al. (2023) leverage queuing models for patient scheduling in healthcare systems, providing insights into optimizing resource allocation while maintaining access constraints, a challenge that also arises in ABAC implementations. Kang et al. (2021) apply a queuing-theoretic model to evaluate transmission risks in confined spaces during pandemics, offering a framework that could be adapted for access request queuing in high-security environments. In addition, Klimenok et al. (2020) study priority-based queuing systems, where access levels dynamically change based on system states — a concept that aligns closely with ABAC policies adapting to contextual attributes. Cheng et al. (2019) introduce a vehicle dispatching queuing model, demonstrating how queuing theory can enhance decision-making in dynamic environments, akin to real-time access control policies in ABAC.

None of the papers surveyed above, however, show how dynamic ABAC systems can be modeled and analyzed as queues, especially where periodic policy reorganization needs to be done along with access rule enforcement. Our paper makes a positive contribution in this direction.

4. Modeling ABAC systems as queues

This section presents our queuing-theoretic framework for representing access request resolution in a dynamic ABAC system. Generally, ABAC functions as an interaction between clients and a server, where clients request access to specific resources, and the server evaluates these requests by comparing entity attributes against the existing policy Π. As more clients send access requests, the server manages an internal queue to handle the incoming traffic. Unlike static systems, where the initially derived policy Π remains fixed, dynamic access control allows new authorizations to be added or removed, updating the initial permission set. These changes to the authorizations are stored in a separate list, referred to as the Auxiliary List. Though there are some proposed approaches to incrementally maintain ABAC policies, these still require an expensive auxiliary structure that can keep growing (Batra et al., 2021). Consequently, making access decisions not only requires evaluation of the policy Π but also checking the Auxiliary List to make sure that no valid authorization exists.

In dynamic ABAC systems initialized with a policy Π, the Auxiliary List can grow due to regular addition of changed authorizations. This growth leads to longer access resolution times, making it necessary to periodically rerun the policy mining algorithm. Since such algorithms are computation intensive, the server must carefully balance the delay in access resolution against the computational cost of policy mining for maximizing the overall system efficiency. To model this scenario, we adopt a queuing-theoretic approach. Specifically, we consider queues with vacations, where the server normally handles access requests by evaluating the ABAC policy and the Auxiliary List, and periodically goes on "vacations" to update the policy by incorporating authorizations from the Auxiliary List. At the end of this vacation, the Auxiliary List is cleared, and the updated policy improves access resolution time until the Auxiliary List grows again.

We primarily consider M/G/1 queues, where the service times have general distribution and the inter-arrival times are exponentially distributed. The rules for server vacations are numerous, with our main focus on comparing an exhaustive service model and an exhaustive-and-limited service model. In an exhaustive service model, the server can take vacations only when the request queue is empty, while in the latter, the server can go on vacation even if the queue is not empty. We describe and compare these two queuing models with varying parameter values.

4.1. M/G/1 Queue with exhaustive service and single vacation model

In the single vacation M/G/1 queuing model with exhaustive service, denoted as (E, SV), the server begins a vacation (i.e., goes for policy rebuilding) immediately after completing service for all access requests in the queue, and returns only when the vacation ends. If there are access requests pending on return, the server resumes access resolution; otherwise, it remains idle until a new request arrives. This system typically models situations where a server takes breaks between busy periods, like machine maintenance or scheduled downtime. Similarly, we model the access resolution procedure on the server side using two exclusive modes as shown in Fig. 1. The first mode is responsible for resolving incoming access requests, while the second focuses on incorporating newly added user-specific permissions from the Auxiliary List into the ABAC policy through policy mining. These server modes are referred to as *Access Resolution* mode and *Policy Mining* mode. When no access requests are waiting to be processed, i.e., when the server queue is empty, the server transitions to the Policy Mining mode. During this time, the access request queue is effectively on *vacation*, meaning any new requests that come in while the server is in Policy Mining mode must wait until the vacation period is over before being processed. Understanding the relationship between various queue parameters is essential for characterizing system performance. The key parameters such as queue length, waiting time and utilization factor are all influenced by the interplay of service time, arrival rate and the vacation period. Some of these are described below (Tian and Zhang, 2006).

- **Access Request Arrival Process**: The access requests received by the server are assumed to follow a Poisson process with rate λ, i.e., the arrival process is Markovian (M).

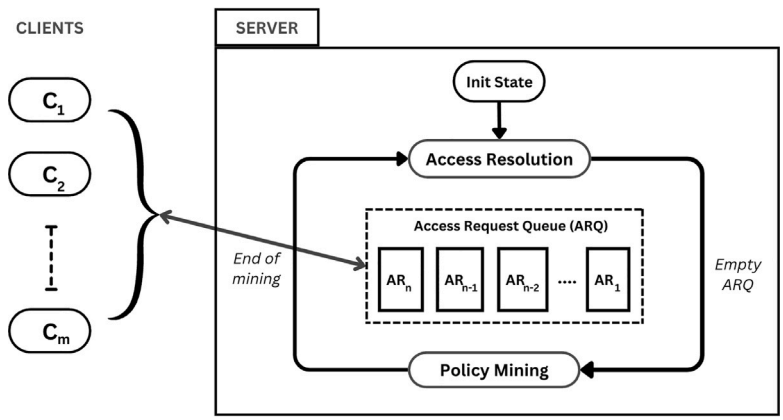

Fig. 1. Queue model for ABAC systems.

- **Access Request Resolution Process**: The time required to resolve an access request during the Access Resolution phase follows a general distribution, as it depends on various factors such as the policy and the length of the Auxiliary List. Hence, the service process for the queue is General (G).
- **Policy Mining Duration**: The duration for each policy mining task is assumed to be independently and identically distributed (iid), with a mean V.
- **Queue Length**: The total queue length Q_t in the system is a sum of two independent components, namely, Q — the queue length of a standard M/G/1 system without vacations and Q_v — the additional queue length due to the vacation effect. Thus, the total queue length for a system with vacations can be expressed as:

$$Q_t = Q + Q_v \tag{1}$$

The standard queue length (Q) has a Probability Generating Function (pgf) given by:

$$Q(z) = (1-\rho)\frac{(1-z)w^*(\lambda(1-z))}{w^*(\lambda(1-z)) - z}, \tag{2}$$

where $\rho = \lambda E(S)$ is the utilization factor, λ is the arrival rate, $w^*(.)$ is the Laplace–Stieltjes Transform (LST) of the service time distribution evaluated at $\lambda(1-z)$ and $E(S)$ is the expected service time. The additional queue length Q_v associated with the vacation effect is described by the following pgf:

$$Q_v(z) = \frac{1 + (1-z)w^*(\lambda) - w^*(\lambda(1-z))}{[w^*(\lambda) + \lambda E(S)](1-z)}, \tag{3}$$

The mean queue length in the ABAC system can be computed by taking the first derivative of the pgfs mentioned in Eqs. (2) and (3) as:

$$E(Q_t) = E(Q) + E(Q_v) \tag{4}$$

- **Waiting Time**: The waiting time of an access request in the system, W_t, is influenced by the vacation, since requests arriving during the server's vacation must wait until the server resumes service. The total waiting time is split into two independent components, namely, W — the waiting time of a standard M/G/1 system without vacations and W_d — the additional waiting time due to the vacation effect. The mean waiting time in the system can be expressed as:

$$E(W_t) = E(W) + E(W_d) \tag{5}$$

where $E(W)$ is the mean waiting time in a standard M/G/1 system (i.e., Without vacation) given by:

$$E(W) = \frac{\lambda E(S^2)}{2(1-\rho)}, \tag{6}$$

where E(S) is the mean service time The additional waiting time $E(W_d)$ due to the vacation can be computed as:

$$E(W_d) = \frac{\lambda E(V^2)}{2[w^*(\lambda) + \lambda E(V)]}, \tag{7}$$

Thus, the total waiting time of an access request is a combination of the standard waiting time (Eq. (6)) and the additional delay caused by the server's vacation (Eq. (7)) performing policy mining.
- **System Time**: The total system time of an incoming access request, T_v, includes both the waiting time and the service time. For the vacation model under consideration, the mean system time is:

$$E(T_v) = E(W_v) + E(V), \tag{8}$$

where $E(V)$ is the mean service time.
- **Server Utilization Factor**: The utilization factor ρ, representing the fraction of time the server is busy, is defined as:

$$\rho = \frac{\lambda E(V)}{1 + \lambda E(S)} \tag{9}$$

where E(S) is the expected service time and E(V) is the expected vacation duration. The additional factor of $\lambda E(V)$ accounts for the time the server spends on vacations. If the server goes on vacations (i.e., it becomes unavailable for some period of time after serving a customer), the total available time for service is reduced.
In this model, utilization is affected by the interplay between service time and the expected vacation duration. When the server is on vacation, it is idle, leading to periods of zero utilization, and when it resumes service, the utilization is fully dependent on the arrival and service rates.
- **Auxiliary List Update**: New user authorizations are added to the Auxiliary List following a Poisson distribution with rate γ.
- **Queue Discipline**: The access request queue follows a First-Come-First-Serve (FCFS) discipline.

In this modeling, we assume that the duration of policy mining (i.e., vacation periods), represented by $V_1, V_2, \ldots$, are independent of both the arrival and resolution of access requests. Each new access request must wait in the queue until all previous requests have been resolved, including those currently being processed or in the queue, or until the policy mining phase finishes. The stochastic nature of the system is preserved by considering residual service times, which are evaluated as the expected remaining time for the completion of either service or vacation when a new access request arrives. The residual service times for an M/G/1 system with vacations is depicted in Fig. 2. Busy periods denoted by X_i alternate with vacation periods denoted by V_i over the lifetime of the ABAC system. Summing the area under Fig. 2 over the system lifetime gives us the total residual time. The mean

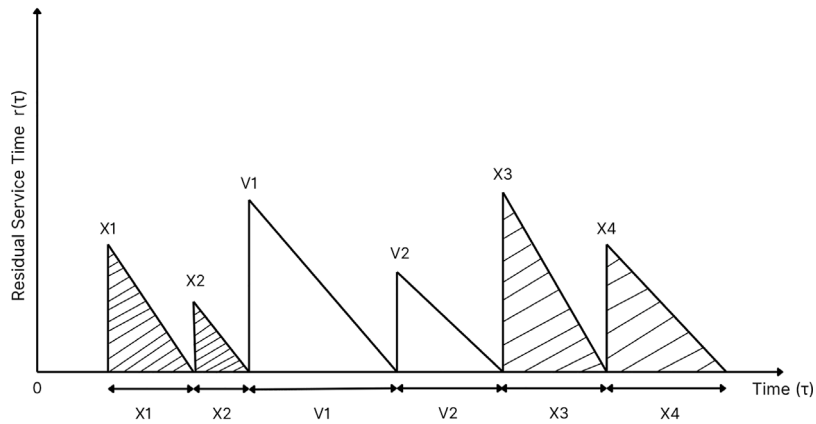

Fig. 2. Residual Service Times for M/G/1 Queue with Vacations.

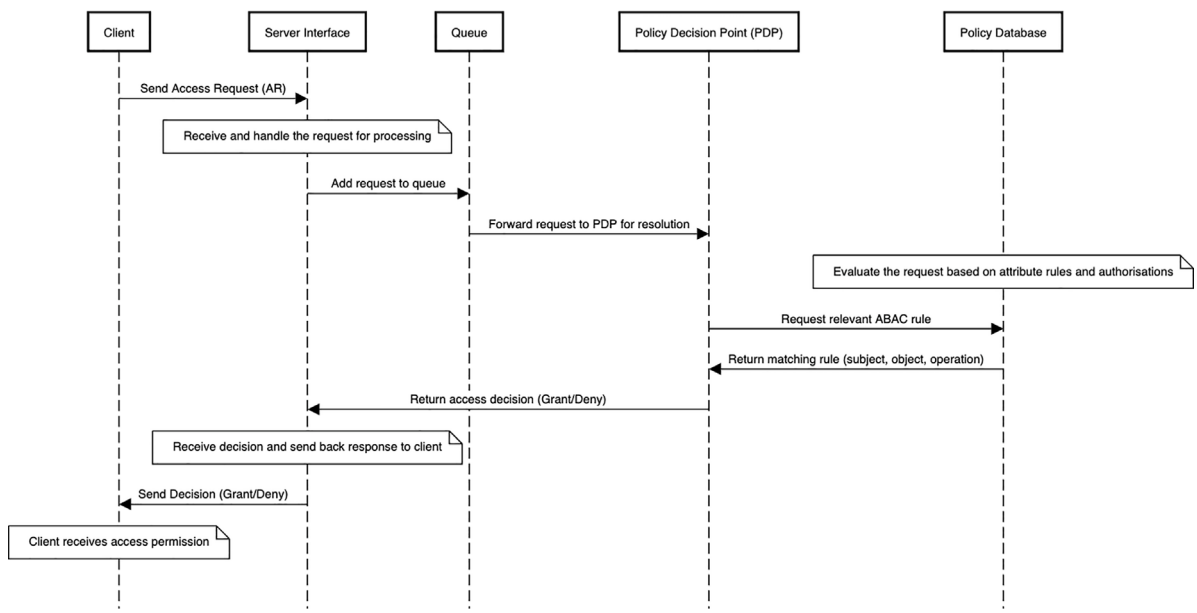

Fig. 3. Access Request Resolution Mode.

residual time, denoted by R, is based on the system's utilization factor ρ and the average vacation duration V (Bertsekas and Gallager, 1992). As time approaches infinity, the average waiting time in the system, denoted by W, is affected by the second moments of both the service times X^2 and the vacation times V^2.

$$W = \frac{\lambda X^2}{2(1-\rho)} + \frac{V^2}{2V}, \tag{10}$$

4.2. M/G/1 Queue with exhaustive and limited service model

In this M/G/1 queuing model (EL, MV), we consider a service system where the server keeps resolving access requests until either:

- A certain number of requests (including those arriving after the service period) are served
- Queue becomes empty

whichever occurs first. This is referred to as the E-limited service, and combines features of both exhaustive and non-exhaustive behavior. Upon returning from vacation, the server resumes serving access requests again. If there are no requests, it remains idle until new arrivals occur. Similar to the previous model, the server exists in two working modes including the *Access Resolution* mode and the *Policy Mining* mode with the only change being in the server vacation policy. The components of the corresponding queue model are as follows.

- **Access Request Arrival Process**: Access requests are received following a Poisson process at a rate λ as seen previously, with the arrival process being Markovian (M).
- **Access Request Resolution Process**: The time to resolve each access request is assumed to follow a general distribution (G).
- **Policy Mining Duration**: The vacation duration is modeled using a general distribution (G). It reflects how long the server spends optimizing policies before returning to handle more requests.
- **Queue Length Decomposition**: In an M/G/1 queue with E-limited service, the total stationary queue length Q_t can be decomposed into three independent components:

$$Q_t = Q + Q_d + Q_r, \tag{11}$$

where Q is the queue length in a standard M/G/1 system as described in Eq. (2), Q_d is the additional queue length due to the vacation effect and Q_r is the additional queue length due to the residual life of the vacation. The pgf of the additional queue length Q_d from the vacation effect is modeled by:

$$Q_d(z) = \frac{1 - w^*(\lambda(1-z))}{\lambda E(V)(1-z)}, \tag{12}$$

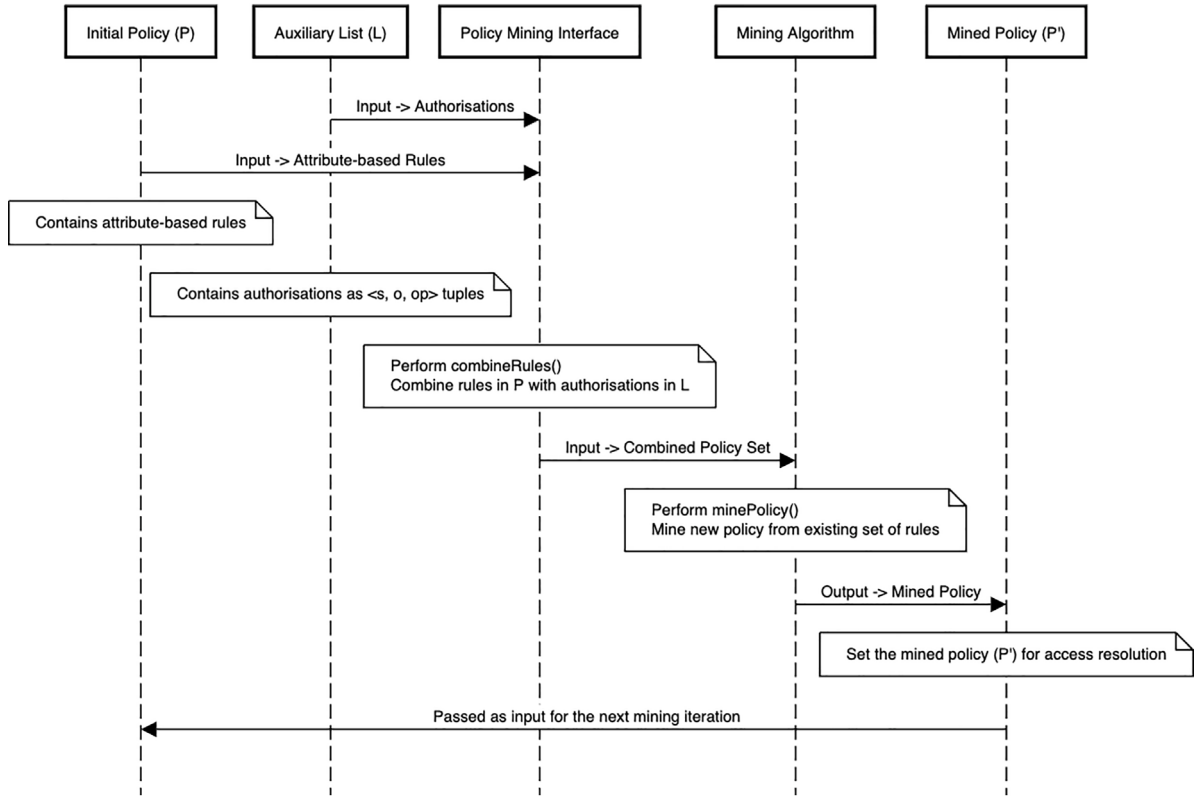

Fig. 4. Policy Mining mode.

where $w^*(.)$ is the LST of the service time and E(V) is the expected vacation duration.

- **Mean Queue Length**: By taking the first derivatives of the generating functions in Eqs. (11), (12), we obtain the mean queue length for each component:

$$E(Q_t) = E(Q) + E(Q_d) + E(Q_r) \tag{13}$$

- **Waiting Time**: The waiting time in the E-limited system can be expressed using Little's Law:

$$E(W_t) = \frac{E(Q_t)}{\lambda}, \tag{14}$$

where $E(W_t)$ is the total waiting time in the system, and λ is the arrival rate. The waiting time comprises the standard M/G/1 waiting time plus delays caused by the vacation and residual effects. Considering the condition $M(1-\rho)-\lambda E(V) > 0$ is satisfied, the total waiting time W_t can be decomposed into a sum of three independent parts similar to the previous case:

$$W_t = W + W_d + W_r \tag{15}$$

By leveraging the E-Limited M/G/1 queue model, the ABAC system efficiently handles access requests during busy periods and refines policies during vacation periods. The combination of service (handling access requests) and vacation (policy mining) ensures that both system performance and policy quality are continually improved. Balancing these two aspects is key to maintaining a robust, scalable, and adaptable dynamic ABAC system.

A sequence diagram depicting the working of the server in the Access Request Resolution Mode is shown in Fig. 3. In this mode, it is responsible for processing each access request that is placed in the queue. Upon receiving an access request, the server forwards it to the Policy Decision Point (PDP) for evaluation against the Attribute-Based Access Control (ABAC) policy Π and the Auxiliary List L, which contains additional authorization tuples. The evaluation involves a comparison between the attributes of the access request (such as the user, object and action as described in Section 2) with those specified in each policy rule. If no matching rule is found in the ABAC policy, the system evaluates the access request against the individual authorization tuples stored in the Auxiliary List L to find an appropriate match. Once a decision is reached (either *Grant* or *Deny*), the result is sent back through the server interface to the client. The client receives the final access decision, which determines whether the requested operation is allowed or denied. This process ensures that access control decisions are made efficiently by leveraging both predefined policy rules and supplemental authorization data.

When the access request queue is emptied, the server transitions into Policy Mining mode, as illustrated in Fig. 1. In this state, the server temporarily suspends access request resolution in order to efficiently run the computationally intensive policy mining algorithm. This phase is crucial for optimizing future access control decisions by updating and refining the system's policies. During policy mining, the server integrates all authorizations from the Auxiliary List L into the existing ABAC policy Π. This merging process, whose sequence diagram is shown in Fig. 4, effectively consolidates the authorizations that were temporarily stored in L during busy periods into a more permanent, structured form within the policy. The updated policy improves the subsequent access resolution process due to a lesser number of rules. This procedure allows the system to evolve its policies over time, incorporating new authorizations and removing outdated ones, which helps maintain an up-to-date security framework for access control. One of the requirements for such a system to work is the assumption of stability, where the rate of incoming requests λ is strictly less than the average service rate μ.

5. System level design for performance analysis

In the previous section, we presented a theoretical discussion of queues with varying vacation policies, along with their use in modeling

dynamic ABAC systems. We now turn our attention to the system-level requirements necessary to effectively use and implement these models for analyzing system performance. Here, we propose an implementation framework that emulates an ABAC system within a practical environment. The system is built on a client–server architecture, focusing on data generation and server operations. The system is divided into programs that run on both the server and client sides. A runner program orchestrates the server and client to perform a set of experiments across various configurations. The codebase is written in Python, utilizing multi-threading, locks and thread events to ensure synchronized data access.

5.1. Server side design

The server side includes a data generation module, a main server program, a PolTree implementation, and the permission database stored in JSON format. PolTree is a tree data structure for efficient evaluation of rules in an ABAC policy (Nath et al., 2019). The main server is structured around three threads: client handler, access request resolver, and auxiliary list updater. The server sets up a socket connection, binds to an address, and waits for client requests.

5.1.1. Data generation module

The data generation module is an independent program designed to create randomized datasets involving policies, users and resources, and is executed before the server and client programs are launched. The dataset is generated with configurable parameters like the number of attributes, values, subjects (users) and objects (resources). These parameters can be adjusted based on the specific requirements of the experiment. Such a flexibility allows testing under various scenarios by varying the number of users, objects and rules involved in the ABAC policy.

In the first step, the module generates a set of attributes and corresponding values for both users and objects. A random combination of uppercase letters is used to create attribute names, while lowercase letters are randomly chosen for the attribute values. Following the generation of attributes, the module creates sets of users and objects. Each user and object is assigned a unique identifier, along with a random combination of attribute–value pairs. To maintain variety, a controlled number of duplicates are allowed so that the generated users and objects exhibit a mix of unique and shared characteristics. The next task is to generate the ABAC policy is represented as a set of low-level rules consisting of user attributes, resource attributes and the type of operation to allow. The module avoids duplicating rules by maintaining a list of already used attribute-value combinations.

5.1.2. Main server

This program serves as the core of the server-side operations. It is structured using multi-threading to ensure synchronized access to shared resources like the access request queue and auxiliary list.

Handle Client thread accepts the client connections. After a successful connection, the server sends user and resource attributes and values to the client. By default, the server operates in *Access Resolution* mode until a client connects. This thread then continues receiving access requests from the said client until the client resets the connection or the server is interrupted.

Access Request Resolver, which is a Policy Decision Point (PDP) thread, handles the core server functions such as resolving access requests during busy times and mining policies during idle periods (Vacations). In each cycle, the vacation condition is checked depending on the vacation model the server decides to either remain in *Access Request Resolution* mode or go on a vacation. While in the *Access Request Resolution* mode, as described in Fig. 3, it processes the access request referring to policy in either of the following mentioned modes, and the auxiliary list. The system is configured to support two vacation models and two modes of access request resolution. The vacation models are:

- **Access Queue Empty:** The server goes to vacation whenever the access request queue is empty (i.e., instantiation of (E,SV) model discussed in Sub- Section 4.1).
- **Access Requests Served:** The server goes to vacation whenever the number of access requests served from the end of the previous vacation reaches a certain threshold or the access request queue is empty (i.e., instantiation of (EL,MV) model discussed in Sub-Section 4.2).

Access request resolution modes:

- **Linear Search:** An access request is matched with each rule in the policy to evaluate if the access can be granted.
- **Policy Tree:** The policy tree is traversed to evaluate if the current access request may be allowed.

The implementation is default access denied, i.e., if an access request is not evaluated as granted by at least one rule, then the output is denied access. After the access request is evaluated through one of the two modes specified above, and there is no supporting rule, it is evaluated using the auxiliary list.

Auxiliary List Updater thread generates new authorization rules and updates the auxiliary list in the server.

The initial ABAC policy, denoted as Π, is represented as a dictionary of rules that include attribute–value pairs and permitted operations. It is created by the data generation module. The auxiliary list, L, is implemented in the form of an in-memory deque that stores user-specific access permissions as tuples as <*subject, object, operation*>. While any policy mining algorithm, could be used, we employ the approach introduced by Xu and Stoller (2015), iterating through the access permissions in an Access Control List, selecting specific tuples to build candidate rules. The rules are then expanded to cover additional access rights from the ACL. Once complete, the rules are consolidated by merging those that encompass all permissions, reducing the policy size.

Assuming the system remains stable, the server eventually reaches a point where no access requests remain in the queue, at which time it switches to the Policy Mining mode. For the second queue model, the server switches to the Policy Mining mode after serving a certain number of access requests even if the queue is not empty. In this mode, the server stops processing access requests and mines a consolidated ABAC policy as shown in Fig. 4. The initial policy, Π, is combined with permissions in the auxiliary list L into a new policy, Π', using the combineRules() method. The refined policy, Π'', is generated by the policy mining algorithm implemented in the minePolicy() method. Finally, access control matrices for the initial and final policies are compared to ensure no unauthorized access remains. This cycle repeats until the client disconnects or the server shuts down.

5.1.3. Policy tree

The policy tree (PolTree) module is responsible for organizing the access control rules in a hierarchical structure to optimize access request evaluations. Each node in the tree represents an attribute from the access control policy, and edges between nodes define possible values or conditions for those attributes. The tree is built utilizing the entropy metric as suggested by Nath et al. (2019).

The `resolve` function checks whether a given access request matches the policy by traversing the PolTree. The traversal starts at the root and descends down through branches that match the attributes in the access request. The tree supports wildcard values for attributes, denoted by '*', allowing flexibility in matching. If a leaf node with the decision *Allow* is reached, the access request is granted; otherwise, it is denied. This hierarchical structure speeds up decision-making during access request evaluations. Further, the policy tree implementation supports dynamic policy updates. New rules can be inserted into the tree, which updates the structure based on the newly defined policy. The system maintains the integrity of the tree by ensuring that policy

updates and insertions are correctly handled to reflect any newly granted permissions. In addition to real-time request handling, the policy tree can be stored and retrieved as a JSON file. These methods allow for efficient persistence of the policy tree structure between system runs, ensuring that the server can quickly resume operations without needing to reinitialize the policy structure.

5.2. Client side design

On the client side, a client program and local attribute-value files generate access requests. The client employs the *select* system call to manage non-blocking I/O operations during communication with the server. The *select* function monitors the socket for any incoming messages from the server, allowing the client to perform other tasks while waiting for responses. This mechanism prevents the client from being blocked by slow network responses or server delays. By using select, the client can efficiently manage multiple I/O streams, waiting for input on the connection without halting its execution. This is particularly useful when simulating high arrival rates of access requests, where the client continuously sends requests and awaits server responses. When select detects that the server has responded (i.e., the socket becomes readable), the client retrieves the data instantaneously. This design ensures that the client can handle multiple concurrent requests while minimizing idle wait times.

The client generates random access requests based on these attribute –value pairs and sends them to the server at random intervals based on an exponential distribution, modeling a Poisson Arrival process. These requests are queued and processed by a Policy Decision Point thread, which matches attribute–value pairs in the requests against the initial policy rules. If no rule matches, the PDP checks permissions in the auxiliary list L. Access decisions are then relayed back to the client via the Policy Enforcement Point (PEP).

6. Experimental results

In this section, we present the results of a comprehensive set of experiments carried out on a CPU-Optimized Digital Ocean Droplet[2] running Ubuntu 24.04 (LTS) operating system, powered by an Intel Xeon Platinum 8280 processor, having 4 GB RAM and 25 GB SSD storage.

We made an extensive search for real-world data that could potentially be used for carrying out our experiments. Unfortunately, there is no such data that is relevant to the kind of studies done in this paper. For meaningful comparison, we specifically need dynamic data — both new authorizations for adding to the Auxiliary List as well as incoming access requests, which are currently unavailable. Organizations are sceptical about sharing access control data, which is considered highly sensitive. In the absence of such real data, our experiments were conducted using synthetic data generated by setting and varying different parameters that result in data distribution as close as possible to real-world data.

To study the performance of the system, we varied four critical parameters, i.e., Initial Policy Size, Auxiliary List (AL) Update Rate, No. of Attributes and Access Request (AR) Arrival Rate as shown in Table 2. The experimental setup also explored queues with varying vacation models and policy evaluation strategies. These variations in the Queuing Model and Policy Evaluation (QM-PE) led to four distinct system configurations, denoted as QM-PE_1, QM-PE_2, QM-PE_3 and QM-PE_4 as shown in Table 3.

In our experimental setup, the number of values corresponding to each attribute was kept constant to ensure uniformity across experiments. Additionally, each reported observation represents the average of metric values recorded over multiple vacations to mitigate potential distortions in the data caused by interfering processes within the system. Since no other existing approach attempts to address the same problem as considered in this paper, there is no baseline for carrying out comparative studies. Instead, we conduct extensive experiments by varying different parameters that can affect the performance of our method.

In the first experiment, we analyze the average access resolution time over multiple vacations as shown in Fig. 5 for five distinct vacation periods. Each bar labeled by V_i corresponds to the average access resolution time before the occurrence of the ith vacation. For V_1, the value is on the higher side, which can be attributed to the larger size of the initial policy. Following this peak, there is a marked decrease in resolution time before the second vacation period *V2*, indicating a substantial reduction in the policy size. Then onwards, the times before each vacation exhibit minimal fluctuation, staying within a narrow range. This stability suggests consistent performance in both the operational modes of the server. The system configuration for this experiment considered a policy consisting of 1000 rules. Each rule is defined with 6 attributes, and each attribute is capable of assuming one of six predefined values. Furthermore, the update rate for the auxiliary list was set at 15 authorizations per second, and the access request arrival rate was established at 80 access requests per second.

For the next few experiments, the number of access requests after which the server goes to vacation is kept at 64 for the (EL, MV) Vacation model. It is observed that for most of the experiment configurations, the condition hits access requests served more frequently than queue empty, while a few configurations have queue empty condition predominantly dictating the vacation condition.

Table 2
Variation in the parameters.

Parameters	v1	v2	v3	v4
Initial Policy Size	100	300	600	1000*
No. of attributes	4	6*	8	10
AR Arrival Rate	5	20	80*	150
AL Update Rate	2	7*	15	20

Note: * Is the default value when that parameter is not varied.

Table 3
Experiment configuration.

Configurations	Queue model	Policy resolution
QM-PE_1	(E, SV)	Linear Search
QM-PE_2	(E, SV)	Policy Tree
QM-PE_3	(EL, MV)	Linear Search
QM-PE_4	(EL, MV)	Policy Tree

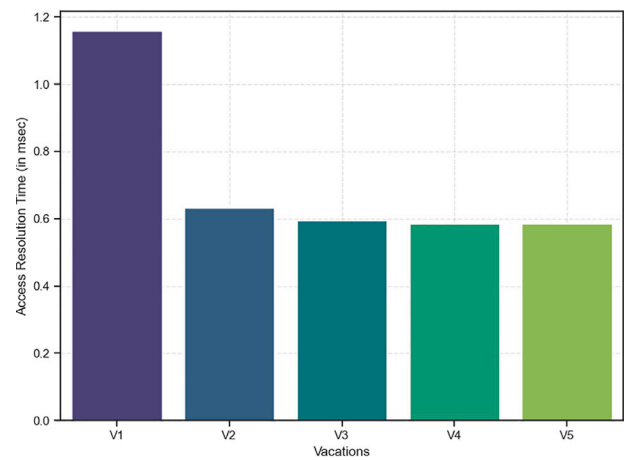

Fig. 5. Access resolution time over multiple vacations.

[2] https://www.digitalocean.com/

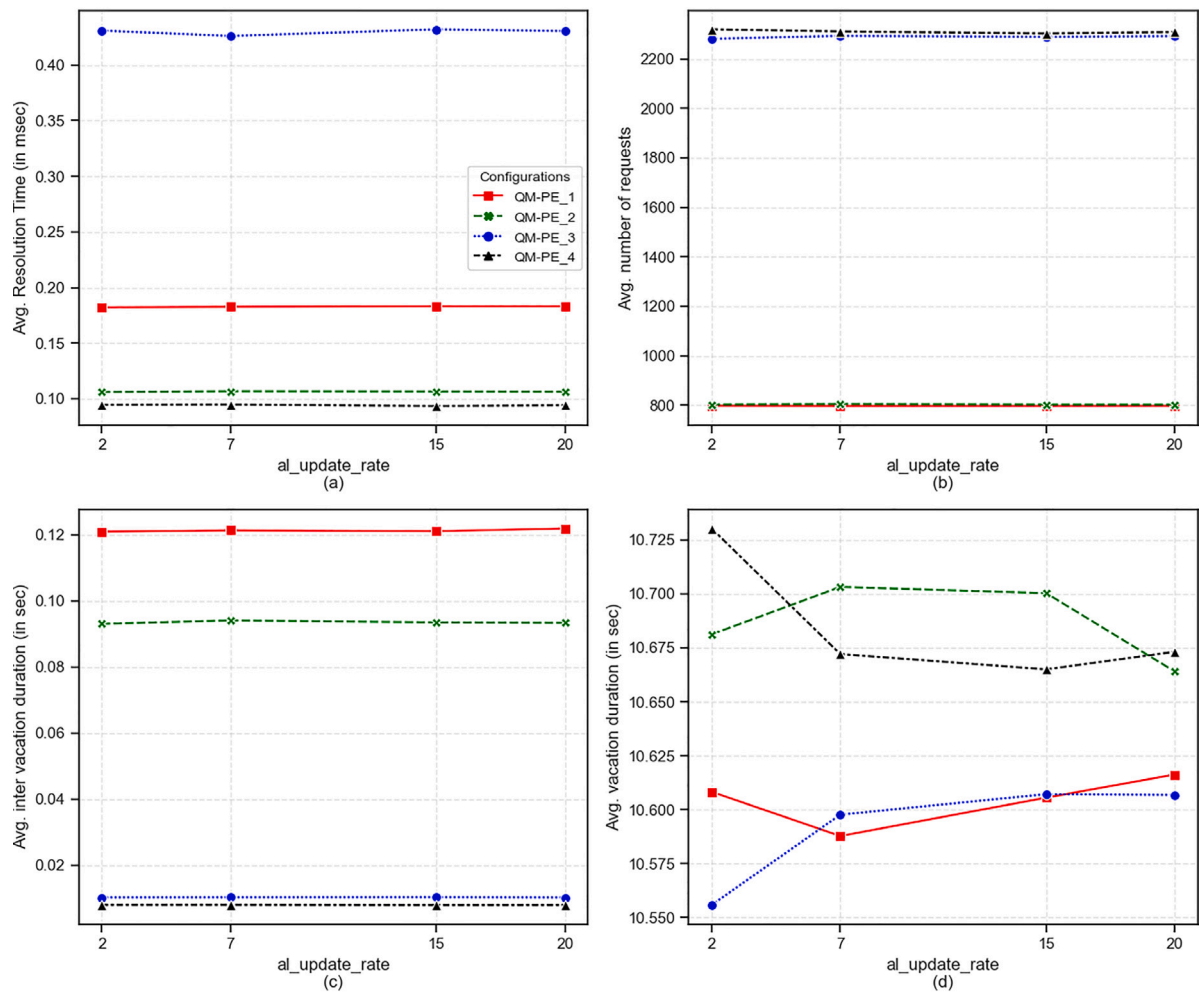

Fig. 6. Variation in performance metrics with auxiliary list update rate.

In the second experiment, we analyze the effect of the auxiliary list update rate on different system parameters as shown in Fig. 6. The first subplot shows the average access resolution time across different AL update rates. QM-PE_3 has the highest resolution time because it uses linear search for policy resolution and triggers vacations based on access requests. This combination may lead to more frequent interruptions and slower searches. On the other hand, QM-PE_1 and QM-PE_2 have lower resolution times, due to their queue empty vacation model, which allows for more efficient processing of requests. The stability across AL update rates suggests that resolution time is more dependent on the queuing model and policy resolution method than the frequency of auxiliary list updates. The second subplot of the same figure reveals the average number of access requests in the system for each configuration. QM-PE_3 and QM-PE_4 show significantly higher numbers of requests, which is due to the more frequent vacations and irregular access request arrival model timings as compared to smoother access requests arrival in case of queue empty model. QM-PE_1 and QM-PE_2, using the queue empty model, process requests more continuously, resulting in fewer accumulated requests. The consistency across AL update rates indicates that the number of requests is primarily influenced by the vacation model rather than update frequency. The third subplot shows the average inter-vacation duration. The inter-vacation durations for configurations QM-PE_1 and QM-PE_2 are stable and significantly higher compared to QM-PE_3 and QM-PE_4. The (E, SV) model in QM-PE_1 and QM-PE_2 leads to longer durations between vacations since the system only triggers a vacation when the request queue is empty. On the other hand, configurations QM-PE_3 and QM-PE_4, which also use number of access requests served as the trigger, show shorter and fluctuating inter-vacation durations because the vacations are reached based on a predefined number of access requests, which can vary depending on system load. Finally, the fourth subplot analyzes the average vacation duration. The variations across configurations and AL update rates are more pronounced here, indicating that vacation duration is sensitive to both system configuration and update frequency. QM-PE_3 and QM-PE_4 generally have longer vacation duration, which aligns with their higher number of accumulated requests. The system might need longer vacations to handle the backlog. The slight fluctuations across AL update rates suggest that more frequent updates (higher rates) might lead to slightly shorter vacations in some configurations, possibly due to more efficient policy management.

In the third experiment, we analyze the effect on system parameters by varying the access request arrival rate as depicted in Fig. 7 in the form of box plots. This type of data representation was used since the experiments showed a significant amount of variation in the observed metrics. The first subplot of the figure examines the impact of arrival rate on the average resolution time across the four configurations. The median values for all configurations remain in a close range, suggesting that resolution time does not vary significantly as the arrival rate changes. However, the resolution times for QM-PE_1 and QM-PE_2 tend to have slightly more variability compared to QM-PE_3 and QM-PE_4, as indicated by the broader interquartile range (IQR) and the presence of more outliers. QM-PE_2 and QM-PE_4 consistently show the lowest resolution time across all arrival rates, which can be attributed to the efficiency of the policy tree resolution mode. In contrast, the linear

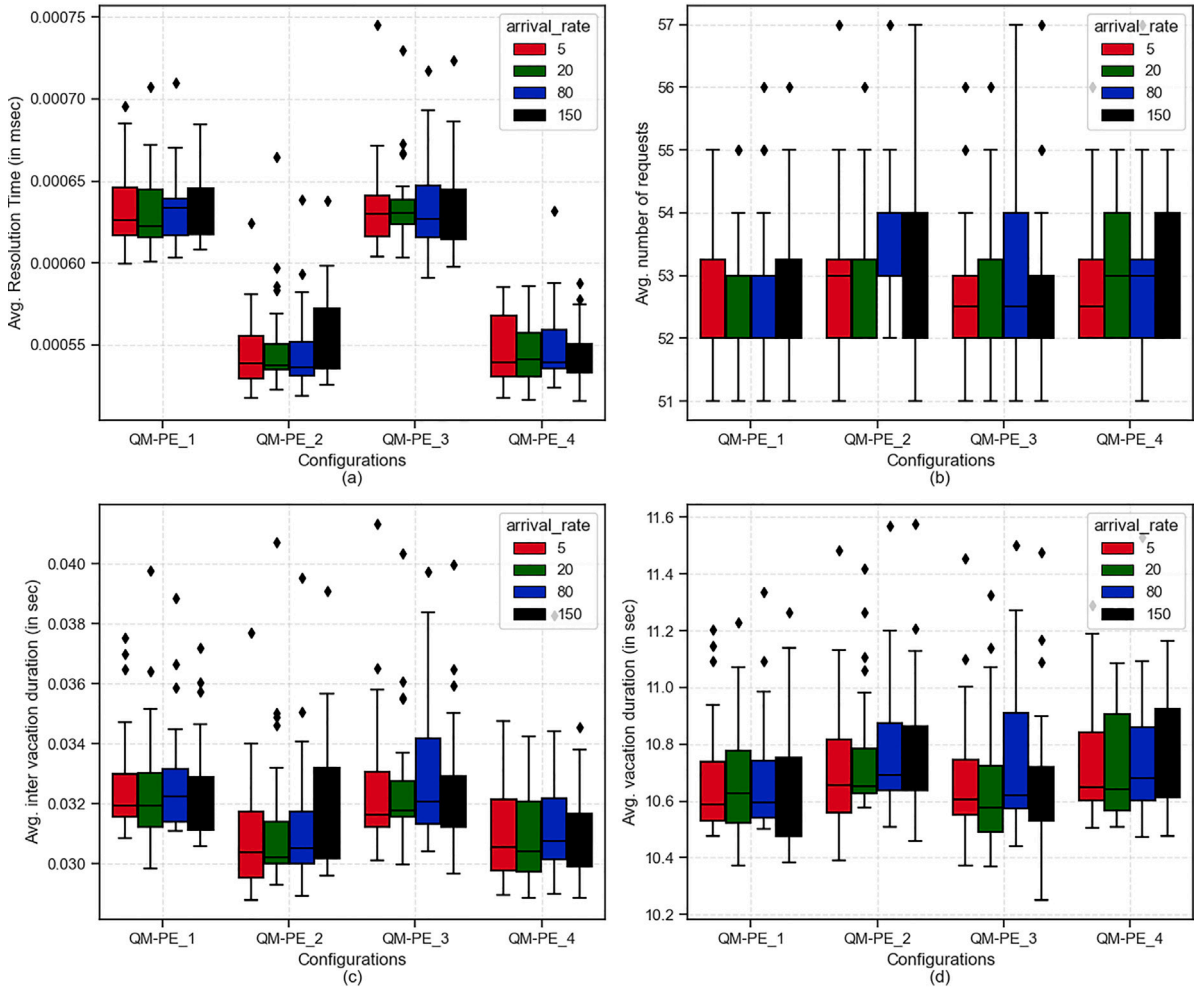

Fig. 7. Variation in performance metrics with access request arrival rate.

search mode in QM-PE_1 and QM-PE_3 contributes to slightly higher resolution times and higher variability of resolution time due to the randomness of finding the granting rule and size of the policy. The second subplot of the same figure shows how the number of access requests in the system changes with the arrival rate. For all configurations, the number of requests in the system increases slightly as the arrival rate increases from 5 to 150, but the overall change is minimal, indicating that the system is capable of handling higher arrival rates without significant degradation in performance. The slight increase with higher arrival rates is logical — more requests are entering the system per unit time. QM-PE_1's stability across arrival rates might indicate that the (E, SV) vacation model allows it to process requests more continuously, preventing large backlogs. QM-PE_3 and QM-PE_4 need to process a marginally higher number of requests compared to QM-PE_1 and QM-PE_2, particularly at higher arrival rates (80 and 150), because of the same reasons as mentioned in the second experiment. This is consistent with the previous observations about access request-based vacations, which allow these configurations to handle more requests before entering a vacation period. The third subplot demonstrates the effect of the arrival rate on the average inter-vacation duration. QM-PE_2's shorter inter-vacation durations as compared to QM-PE_1 stems from its efficient policy tree resolution, allowing it to process requests quickly and trigger vacations more frequently, similarly QM-PE_4 has shorter inter-vacation duration compared to QM-PE_3. The more number of outliers in QM-PE_1 and QM-PE_3 are attributed to the randomness of the liner search policy evaluation mode as discussed previously for the first subplot. The fourth subplot investigates the average vacation duration across different arrival rates. For all configurations, the vacation duration remains relatively stable across the change in the arrival rates, with minor fluctuations. Longer vacation duration for QM-PE_3 and QM-PE_4 are observed because Access Request Vacation model might accumulate more work to be done during vacations. The expected trend of increasing vacation duration with higher arrival rates is followed in QM-PE_1 and QM-PE_2 as the number of access requests per second increases, it would take longer for the vacation condition to be reached in case of (E, SV) model, thus the length of the auxiliary list increases.

In the fourth experiment, we analyze the impact on system parameters by varying the number of attributes as shown in Fig. 8. The first subplot of the figure depicts how the average resolution time is influenced by the number of attributes across all four configurations. As the number of attributes increases, the resolution time shows a slight increase for all configurations except QM-PE_4, which remains more stable. The linear search-based configurations, QM-PE_1 and QM-PE_3, exhibit higher average resolution times overall, with QM-PE_1 having the highest resolution time. The use of a policy tree in QM-PE_4 results in the lowest resolution times, as the policy tree evaluation is more efficient in handling a larger number of attributes compared to linear search, which becomes less efficient as the attribute space grows. While QM-PE_2 at a first glance might look like an anomaly, the average resolution times are generally less for the (EL, MV) model because of the lower accumulated time of the access requests in the queue. The vacation is reached faster in that model and after the vacation, the queued requests have already waited for the *vacation duration* time. This multiplies over the number of requests per inter-vacation, and accumulates more for the Queue Empty model as compared to

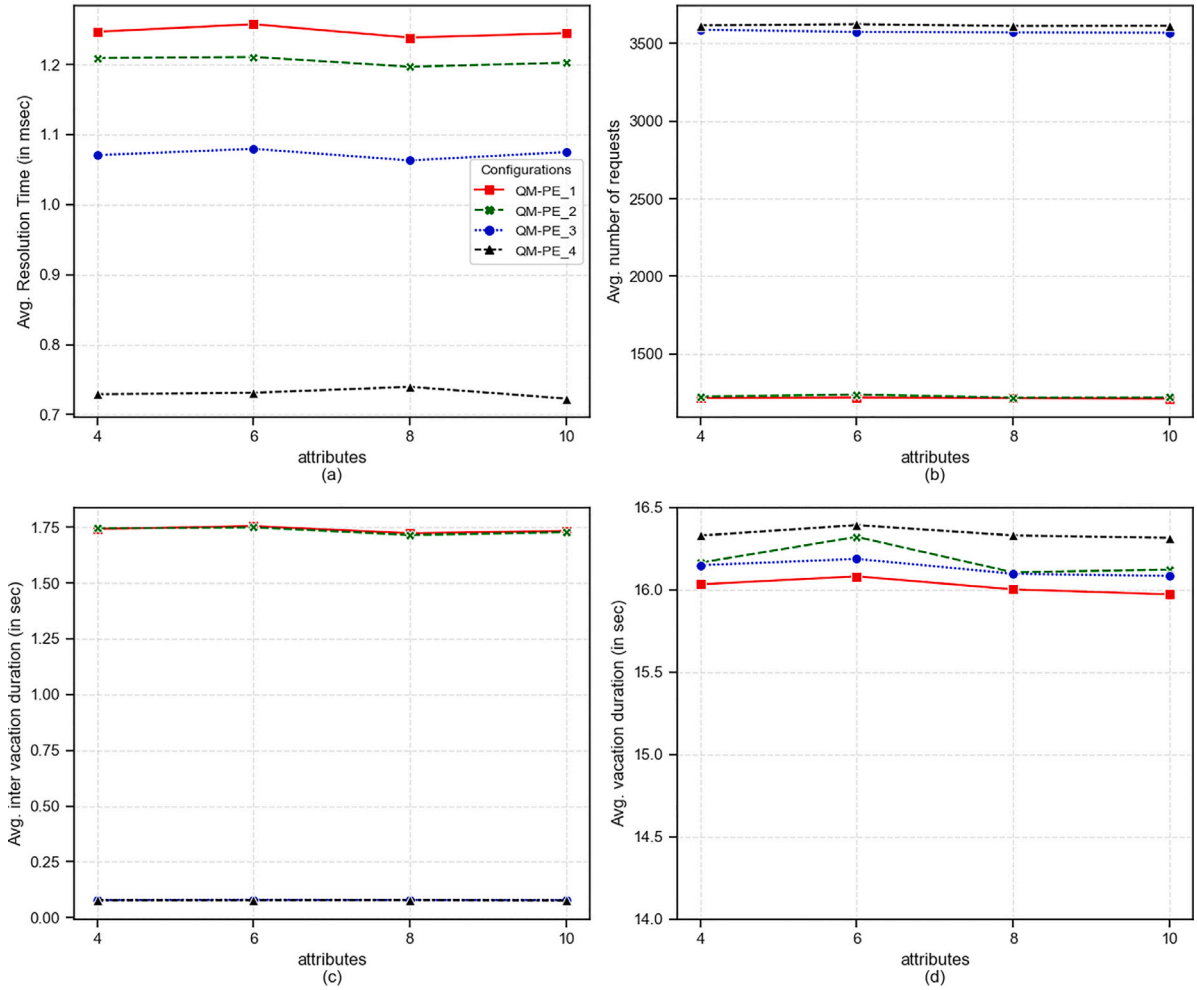

Fig. 8. Variation in performance metrics with number of attributes.

Access requests vacation model. The second subplot of the same figure demonstrates the effect of the number of attributes on the average number of requests in the system by each configuration. QM-PE_3 and QM-PE_4 are consistently cluttered with the highest number of requests in the system, with both configurations processing around 3500 requests regardless of the number of attributes. In contrast, QM-PE_1 and QM-PE_2 handle significantly fewer requests, with the number remaining around 1500 for all attribute values. This is due to the (E, SV) vacation model, which processes requests more conservatively as mentioned in the second experiment. The third subplot shows the average inter-vacation duration as a function of the number of attributes. The inter-vacation duration remains almost constant across all configurations, except for QM-PE_4, where there is a very slight reduction as the number of attributes increases. This suggests that the number of attributes has little effect on the time between vacation periods for most configurations, though the policy tree in QM-PE_4 may become slightly more efficient as the attribute count grows. The fourth subplot shows how the average vacation duration changes with the number of attributes. QM-PE_4 having the longest vacation durations is likely due to its combination of policy tree structure and (EL, MV) model. The policy tree might require more complex updates during vacations, especially with a potentially larger backlog of requests. QM-PE_2 and QM-PE_3 having similar durations suggests that the benefits of the policy tree structure in QM-PE_2 are somewhat offset by the (E, SV) model, while the linear search in QM-PE_3 is compensated by its (EL, MV) model. QM-PE_1 having the shortest durations could be due to its simpler linear search structure combined with the (E, SV) model, possibly resulting in less complex update operations during vacations. The small range of vacation durations (16.0 to 16.5 s) across all configurations suggests that the system is well-optimized overall. The vacation processes seem to be designed to maintain consistent performance regardless of the specific configuration or number of attributes.

In the final experiment, we analyze the impact of varying the initial policy size on the system's performance, as illustrated in Fig. 9. The policy size is varied between 100 and 1000, and its influence on system configurations (QM-PE_1, QM-PE_2, QM-PE_3, and QM-PE_4) is analyzed in terms of resolution time, number of processed requests, and vacation behaviors. The first subplot demonstrates how policy size affects average resolution time across different configurations. As anticipated, resolution time increases slightly with policy size for all configurations due to the increased complexity of larger policies. QM-PE_3, employing linear search for policy resolution, consistently shows the highest resolution times. This is because linear search becomes less efficient as the policy size grows. Conversely, QM-PE_4, utilizing a policy tree, exhibits the lowest resolution times, demonstrating the superior scalability of tree-based structures for larger policies. QM-PE_1 and QM-PE_2 display moderate resolution times, with QM-PE_1's linear search resulting in slightly higher times compared to the tree-based QM-PE_2. The second subplot illustrates the impact of policy size on the number of requests in the system by each configuration. QM-PE_3 and QM-PE_4 need to accommodate significantly more requests (nearly 2400 on average) compared to QM-PE_1 and QM-PE_2 (around 800 requests). This stark difference is attributed to their distinct vacation

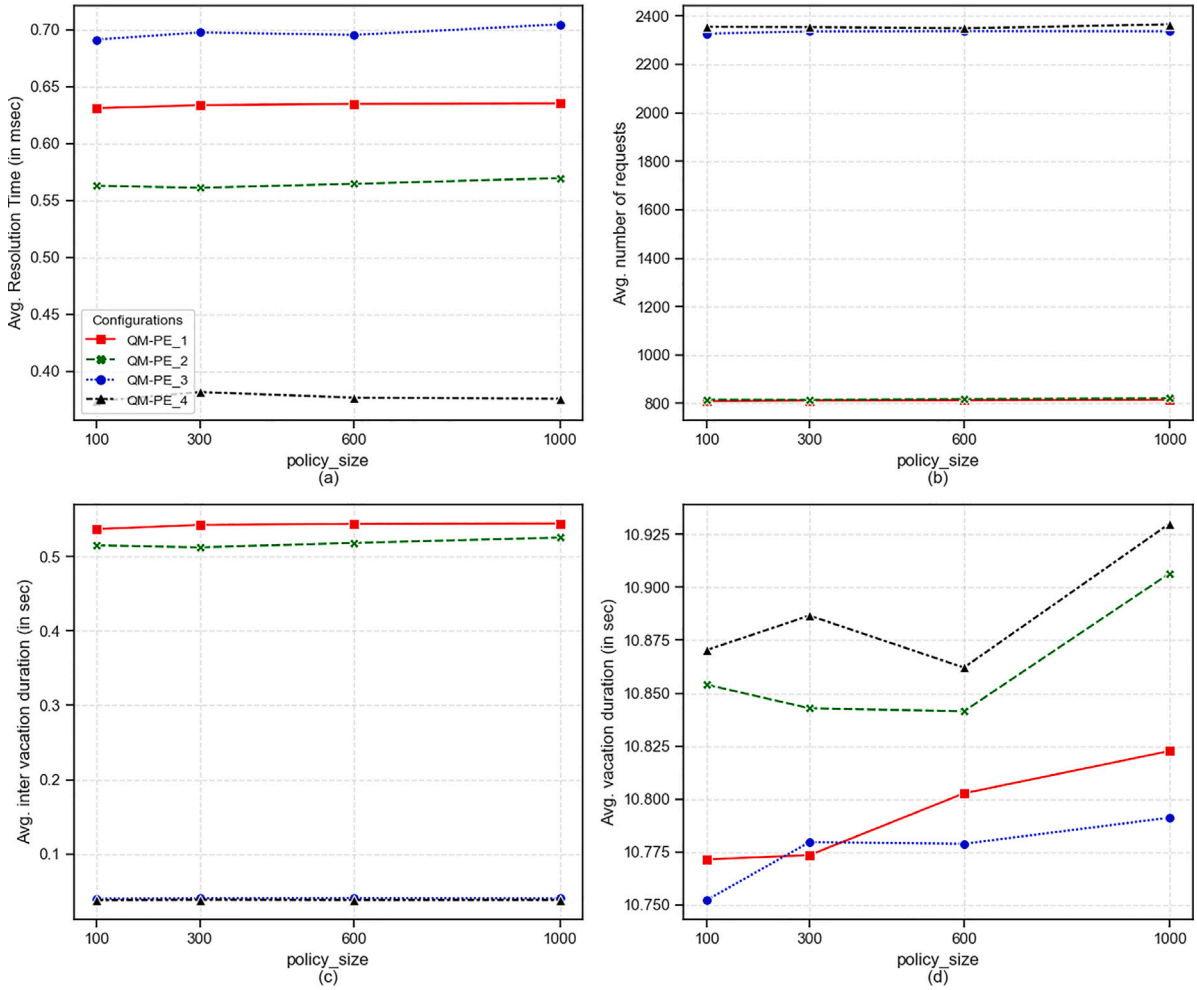

Fig. 9. Variation in performance metrics with auxiliary list update rate.

models: QM-PE_3 and QM-PE_4 trigger vacations based on the number of access requests, while QM-PE_1 and QM-PE_2 enter vacation periods only when the queue is empty as discussed in . Notably, the number of requests remains relatively constant across policy sizes for all configurations, indicating that this metric is more influenced by the vacation model than policy complexity. The third subplot examines how average inter-vacation duration varies with policy size. QM-PE_1 and QM-PE_2 maintain stable inter-vacation durations around 0.5 s, while QM-PE_3 and QM-PE_4 exhibit much shorter, consistent inter-vacation durations. This pattern emphasizes that inter-vacation duration is primarily determined by the chosen vacation model rather than policy size. The (E, SV) model (QM-PE_1 and QM-PE_2) results in longer working periods before vacations, while the (EL, MV) model (QM-PE_3 and QM-PE_4) leads to more frequent, shorter processing intervals. The fourth subplot shows how average vacation duration changes with policy size. All configurations display an increase in vacation duration as policy size grows, reflecting the additional time required for updating or maintaining larger policies during vacations. QM-PE_4 exhibits the most pronounced increase, especially as the policy size approaches 1000, followed by QM-PE_3. This suggests that configurations using more complex data structures (like policy trees) may require more extensive restructuring during vacations as policies grow. QM-PE_1 and QM-PE_2, likely using simpler linear structures, show more gradual increases in vacation duration with policy size growth.

In summary, the following conclusions can be drawn from our experiments about the relationship between the system parameters and the performance metrics:

1. System configuration significantly impacts performance:
   - Configurations using policy trees (QM-PE_2, QM-PE_4) consistently outperform those using linear search (QM-PE_1, QM-PE_3) in terms of resolution time and scalability.
   - Vacation models ((E, SV) vs. (EL, MV)) affect the number of accumulated requests and inter-vacation durations.
2. Initial policy size directly influences multiple performance metrics:
   - Larger policies lead to increased resolution times and vacation durations.
   - Policy tree method demonstrates better scalability with increasing policy size.
3. Number of attributes affects system performance:
   - Slight increase in resolution time for most configurations as attributes increase.
   - Policy tree method (QM-PE_4) shows better stability with increasing attributes.
4. Access Request (AR) arrival rate impacts system behavior:
   - Higher arrival rates lead to slightly more requests in the system.
   - System demonstrates good scalability, handling increased arrival rates without significant performance degradation.

5. Auxiliary List (AL) update rate has subtle effects:
   - Minimal impact on resolution time and number of requests.
   - Slightly affects vacation durations, with higher rates potentially leading to shorter vacations.
6. Vacation behavior is influenced by multiple factors:
   - Vacation durations are sensitive to system configuration and policy complexity.
   - (EL, MV) vacation model generally leads to longer vacation durations.

It may be noted that studies similar to the ones done by us can be made by security administrators for their own ABAC systems. This would enable them to make an informed decision on whether to upgrade the server configuration, design and implement a more efficient policy mining algorithm or a better policy evaluation algorithm, discourage addition of ad hoc authorizations in the Auxiliary List, or increase the inter vacation time to delay the policy rebuilding process.

7. Conclusions

In this paper, we have addressed the problem of performance analysis for access control enforcement in ABAC, considering the dynamic nature of permission updates and the complexity of policy mining. By modeling the system as a queue with vacation, we have investigated the interplay between access mediation and policy mining. Two approaches have been considered, namely, *M/G/1 queues with exhaustive service and vacation* model and *M/G/1 queues with exhaustive and limited service* model. Through extensive experiments over a wide range of synthesized datasets, we have explored the impact of key system parameters, including request arrival rate, access resolution time, vacation duration, and the interval between vacations. The results show that higher emphasis should be given on efficient policy evaluation and choice of vacation discipline. Besides, efficiency of policy mining algorithms has a significant impact on the working of dynamic ABAC systems. Security administrators should, therefore, focus on these aspects while tuning their ABAC instances for addressing any performance bottleneck.

There is still some scope for improvement in our work. As noted in Section 6, the experiments were carried out with simulated data. Even though all attempts were made to ensure that the simulated data is as close as possible to real data, it would be good to repeat the experiments with data actually generated in an organization as and when they become available. Deploying our queuing theoretic framework in an organization and checking its performance is yet to be done.

Future work involves studying the impact of various policy mining algorithms and experimenting with even larger datasets. Another interesting direction would be to see how the scheme behaves under other queue models. This work assumes that policy enforcement and policy rebuilding run on the same computational resources. In an alternative model, the access resolution process could continue to run on a subset of the threads while policy rebuilding is done on the remaining ones. This would cause a partial degradation of access resolution during policy rebuilding but not a complete stoppage. Queues with synchronous vacation with some servers can be employed to model such a scenario. Another option is to make the server go on vacation when the Auxiliary List size increases beyond a certain length or when the queue is empty.

CRediT authorship contribution statement

Gaurav Madkaikar: Writing – original draft, Software, Investigation, Methodology, Data Curation . **Karthikeya S.M. Yelisetty:** Writing – original draft, Software, Investigation, Methodology, Data curation . **Shamik Sural:** Writing – original draft, Supervision, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Jaideep Vaidya:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization. **Vijayalakshmi Atluri:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The research work reported in this publication was supported by the National Institutes of Health, United States under award R35GM134927, by the National Science Foundation, United States under award CNS-2333225 and by Cisco award number 2020-220329 (3696). The content is solely the responsibility of the authors and does not necessarily represent the official views of the agencies funding the research.

Data availability

Data will be made available on request.

References

Abouzaid, L., Sabir, E., Errami, A., Elbiaze, H., 2019. A queuing theoretic framework for flying mesh network assisted IoT environments. In: 2019 IEEE 5th World Forum Internet Things (WF- IoT). pp. 882–887.

Abu Jabal, A., Bertino, E., Lobo, J., Law, M., Russo, A., Calo, S., Verma, D., 2020. Polisma-a framework for learning attribute-based access control policies. In: 25th European Symposium on Research in Computer Security. pp. 523–544.

Aggarwal, N., Sural, S., 2023. Ransam: Randomized search for ABAC policy mining. In: ACM Conference on Data and Applciation Security and Privacy. pp. 291—-293.

Ala, A., Yazdani, M., Ahmadi, M., Poorianasab, A., Attari, M.Y.N., 2023. An efficient healthcare chain design for resolving the patient scheduling problem: queuing theory and MILP-ASA optimization approach. Ann. Oper. Res. 1–31.

Alohaly, M., Balogun, O., Takabi, D., 2022. Integrating cyber deception into attribute-based access control (ABAC) for insider threat detection. IEEE Access 10, 108965–108978.

Bakhtiary, V., Mirabi, M., Salajegheh, A., Erfani, S.H., 2024. Combo-chain: Towards a hierarchical attribute-based access control system for IoT with smart contract and sharding technique. Internet Things 25, 101080.

Batra, G., Atluri, V., Vaidya, J., Sural, S., 2019. Policy reconciliation and migration in attribute based access control. In: Garg, D., Kumar, N.V.N., Shyamasundar, R.K. (Eds.), Information Systems Security. Springer International Publishing, Cham, pp. 99–120.

Batra, G., Atluri, V., Vaidya, J., Sural, S., 2021. Incremental maintenance of ABAC policies. In: Proceedings of the Eleventh ACM Conference on Data and Application Security and Privacy. CODASPY '21, Association for Computing Machinery, New York, NY, USA, pp. 185–196.

Bertsekas, D., Gallager, R., 1992. Data Networks. Prentice-Hall, Inc., USA.

Burmester, M., Magkos, E., Chrissikopoulos, V., 2013. T-ABAC: An attribute-based access control model for real-time availability in highly dynamic systems. 2013 IEEE Symp. Comput. Commun. (ISCC) 000143–000148.

Cheng, P., Feng, C., Chen, L., Wang, Z., 2019. A queueing-theoretic framework for vehicle dispatching in dynamic car-hailing. 2019 IEEE 35th Int. Conf. Data Eng. (ICDE) 1622–1625.

Das, S., Sural, S., Vaidya, J., Atluri, V., 2018a. Hype: A hybrid approach toward policy engineering in attribute-based access control. IEEE Lett. Comput. Soc. 1 (2), 25–29.

Das, S., Sural, S., Vaidya, J., Atluri, V., 2018b. Using gini impurity to mine attribute-based access control policies with environment attributes. In: Proceedings of the 23nd ACM Symposium on Access Control Models and Technologies. pp. 213–215.

Doshi, B.T., 1986. Queueing systems with vacations—a survey. Queueing Syst. Theory Appl. 1 (1), 29–66.

Gautam, M., Jha, S., Sural, S., Vaidya, J., Atluri, V., 2017. Poster: Constrained policy mining in attribute based access control. In: Proceedings of the 22nd ACM on Symposium on Access Control Models and Technologies. pp. 121–123.

Goyal, G., Liu, P., Sural, S., 2022. Securing smart home IoT systems with attribute-based access control. In: ACM Workshop on Secure and Trustworthy Cyber-Physical Systems. pp. 37–46.

Gupta, E., Sural, S., Vaidya, J., Atluri, V., 2023. Enabling attribute-based access control in NoSQL databases. IEEE Trans. Emerg. Top. Comput. 11 (1), 208–223.

Harchol-Balter, M., 2013. Performance Modeling and Design of Computer Systems: Queueing Theory in Action, first ed. Cambridge University Press, USA.
Harrison, M.A., Ruzzo, W.L., Ullman, J.D., 1976. Protection in operating systems. Commun. ACM 19 (8), 461–471.
Hu, V.C., Ferraiolo, D., Kuhn, R., Friedman, A.R., Lang, A.J., Cogdell, M.M., Schnitzer, A., Sandlin, K., Miller, R., Scarfone, K., et al., 2013. Guide to attribute based access control (abac) definition and considerations (draft). NIST Spec. Publ. 800 (162), 1–54.
Iyer, P., Masoumzadeh, A., 2018. Mining positive and negative attribute-based access control policy rules. In: ACM Symposium on Access Control Models and Technologies. pp. 161–172.
Jabal, A.A., Bertino, E., Lobo, J., Verma, D.C., Calo, S.B., Russo, A., 2023. FLAP - a federated learning framework for attribute-based access control policies. In: Shehab, M., Fernández, M., Li, N. (Eds.), Proceedings of the Thirteenth ACM Conference on Data and Application Security and Privacy, CODASPY 2023, Charlotte, NC, USA, April 24-26, 2023. ACM, pp. 263–272.
Kang, K., Doroudi, S., Delasay, M., Wickeham, A., 2021. A queueing-theoretic framework for evaluating transmission risks in service facilities during a pandemic. Prod. Oper. Manage. 32, 1453–1470.
Kaur, J., Rani, R., Kalra, N., 2024. Attribute-based access control scheme for secure storage and sharing of EHRs using blockchain and IPFS. Clust. Comput. 27 (1), 1047–1061.
Klimenok, V.I., Dudin, A.N., Dudina, O.S., Kochetkova, I.A., 2020. Queuing system with two types of customers and dynamic change of a priority. Mathematics.
Kumar, R., Soodan, B.S., Kuaban, G.S., Czekalski, P., Sharma, S., 2021. Performance analysis of a cloud computing system using queuing model with correlated task reneging. J. Physics: Conf. Ser. 2091.
Ma, F.-Q., Fan, R.-N., 2022. Queuing theory of improved practical Byzantine fault tolerant consensus. Mathematics.
Madkaikar, G., Sural, S., Vaidya, J., Atluri, V., 2024. Queuing theoretic analysis of dynamic attribute-based access control systems. In: Pitropakis, N., Katsikas, S., Furnell, S., Markantonakis, K. (Eds.), ICT Systems Security and Privacy Protection. Springer Nature Switzerland, Cham, pp. 323–337.
Mahalingam, N., Sharma, P., 2024. Secure monitoring model for smart agriculture using an optimized attribute-based access control centralized authority system. Multim. Tools Appl. 83 (15), 44781–44798.
Mas, L., Vilaplana, J., Mateo, J., Solsona, F., 2022. A queuing theory model for fog computing. J. Supercomput. 78, 11138–11155.
Menasce, D., 2000. Scaling for e-business. In: IEEE International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunications Systems. pp. 511–513.
Mocanu, D., Turkmen, F., Liotta, A., 2015. Towards ABAC policy mining from logs with deep learning. In: Intelligent Systems.
Morisset, C., Willemse, T., Zannone, N., 2019. A framework for the extended evaluation of ABAC policies. Cybersecurity 2.
Nath, R., Das, S., Sural, S., Vaidya, J., Atluri, V., 2019. PolTree: A data structure for making efficient access decisions in ABAC. Proc ACM Symp Access Control. Model. Technol 2019, 25–35.
Paul, P., Sural, S., 2021. Towards efficient evaluation of ABAC policies using high-dimensional indexing techniques. In: Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications. pp. 243–251.
Perez-Haro, A., Diaz-Perez, A., 2024. ABAC policy mining through affiliation networks and biclique analysis. Information 15 (1), 45.
Sandhu, R., 1993. Lattice-based access control models. Computer 26 (11), 9–19.
Sandhu, R., Coyne, E., Feinstein, H., Youman, C., 1996. Role-based access control models. Computer 29 (2), 38–47.
Schlegel, M., Amthor, P., 2021. The missing piece of the ABAC puzzle: A modeling scheme for dynamic analysis. In: International Conference on Security and Cryptography.
Shang, S., Wang, X., Liu, A., 2024. ABAC policy mining method based on hierarchical clustering and relationship extraction. Comput. Secur. 139, 103717.
Shortle, J.F., Thompson, J.M., Gross, D., Harris, C.M., 2018. Fundamentals of queueing theory, vol. 399, John Wiley & Sons.
Son, H.X., Tran, L.K., Dang, T.K., Pham, Y.N., 2016. Rew-XAC: An approach to rewriting request for elastic ABAC enforcement with dynamic policies. 2016 Int. Conf. Adv. Comput. Applications (ACOMP) 25–31.
Stewart, W.J., 2009. Probability, Markov Chains, Queues, and Simulation: The Mathematical Basis of Performance Modeling. Princeton University Press, USA.
Takagi, H., 1991. Queueing Analysis: Discrete-time systems. Queueing Analysis: A Foundation of Performance Evaluation, North-Holland.
Talegaon, S., Batra, G., Atluri, V., Sural, S., Vaidya, J., 2022. Contemporaneous update and enforcement of ABAC policies. In: Proceedings of the 27th ACM on Symposium on Access Control Models and Technologies. SACMAT '22, Association for Computing Machinery, New York, NY, USA, pp. 31–42.
Talukdar, T., Batra, G., Vaidya, J., Atluri, V., Sural, S., 2017. Efficient bottom-up mining of attribute based access control policies. In: 2017 IEEE 3rd International Conference on Collaboration and Internet Computing. CIC, IEEE, pp. 339–348.
Tian, N., Zhang, Z.G., 2006. Vacation queueing models theory and applications. Oper. Res. Manag. Sci..
Trenchev, I., Dimitrov, W., Dimitrov, G., Ostrovska, T., Trencheva, M., 2023. Mathematical approaches transform cybersecurity from protoscience to science. Appl. Sci..
Uzun, E., Atluri, V., Sural, S., Vaidya, J., Parlato, G., Ferrara, A.L., Parthasarathy, M., 2012. Analyzing temporal role based access control models. In: Proceedings of the 17th ACM Symposium on Access Control Models and Technologies. pp. 177–186.
Xia, Y., Zhai, S., Wang, Q., Hou, H., Wu, Z., Shen, Q., 2022. Automated extraction of ABAC policies from natural-language documents in healthcare systems. 2022 IEEE Int. Conf. Bioinform. Biomedicine (BIBM) 1289–1296.
Xu, Z., Stoller, S.D., 2015. Mining attribute-based access control policies. IEEE Trans. Dependable Secur. Comput. 12 (5), 533–545.